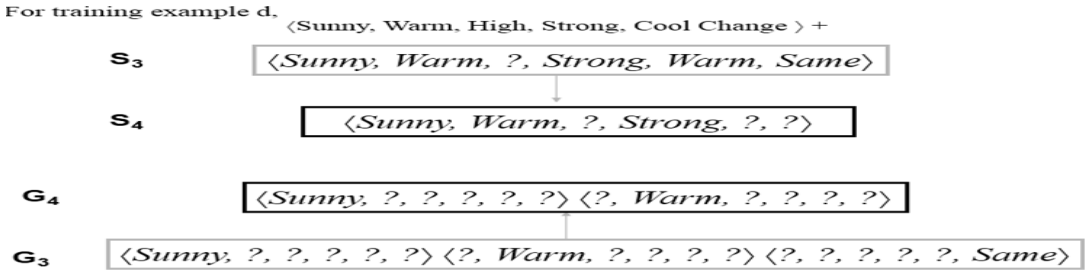
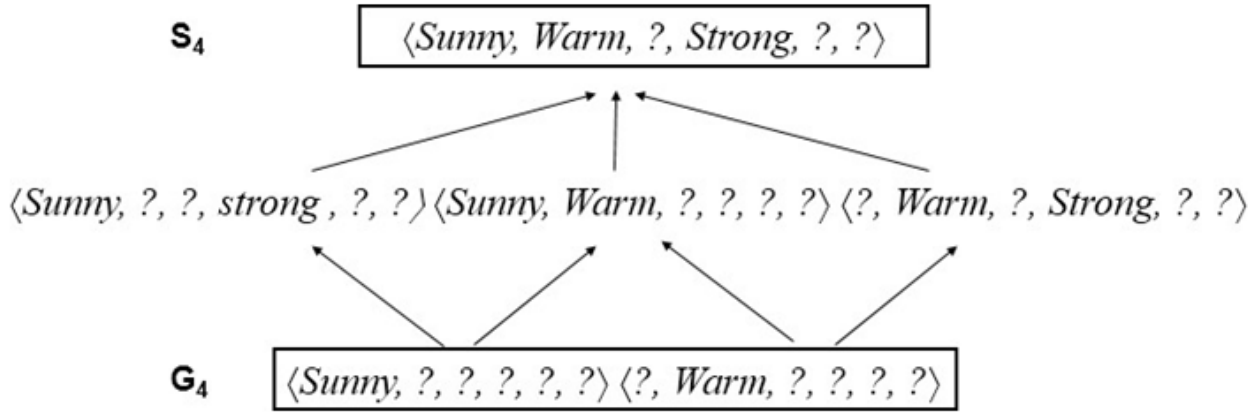


- This positive example further generalizes the S boundary of the version space. It also results in removing one member of the G boundary, because this member fails to cover the new positive example

After processing these four examples, the boundary sets S4 and G4 delimit the version space of all hypotheses consistent with the set of incrementally observed training examples.



1.8 Probably approximately correct learning

In computer science, computational learning theory (or just learning theory) is a subfield of artificial intelligence devoted to studying the design and analysis of machine learning algorithms. In computational learning theory, probably approximately correct learning (PAC learning) is a framework for mathematical analysis of machine learning algorithms. It was proposed in 1984 by Leslie Valiant.

In this framework, the learner (that is, the algorithm) receives samples and must select a hypothesis from a certain class of hypotheses. The goal is that, with high probability (the "probably" part), the selected hypothesis will have low generalization error (the "approximately correct" part). In this section we first give an informal definition of PAC-learnability. After introducing a few nore notions, we give a more formal, mathematically oriented, definition of PAC-learnability. At the end, we mention one of the applications of PAC-learnability.

PAC-learnability

To define PAC-learnability we require some specific terminology and related notations.

- Let X be a set called the instance space which may be finite or infinite. For example, X may be the set of all points in a plane.
- A concept class C for X is a family of functions c : X → {0; 1}. A member of C is called a concept. A concept can also be thought of as a subset of X. If C is a subset of X, it defines a unique function μ_c : X → {0; 1} as follows:

$$\mu_C(x) = \begin{cases} 1 & \text{if } x \in C \\ 0 & \text{otherwise} \end{cases}$$

- A hypothesis h is also a function h : X → {0; 1}. So, as in the case of concepts, a hypothesis can also be thought of as a subset of X. H will denote a set of hypotheses.
- We assume that F is an arbitrary, but fixed, probability distribution over X.
- Training examples are obtained by taking random samples from X. We assume that the samples are randomly generated from X according to the probability distribution F.

Now, we give below an informal definition of PAC-learnability.

Definition (informal)
Let X be an instance space, C a concept class for X, h a hypothesis in C and F an arbitrary, but fixed, probability distribution. The concept class C is said to be PAC-learnable if there is an algorithm A which, for samples drawn with any probability distribution F and any concept c Є C, will with high probability produce a hypothesis h Є C whose error is small.

Examples

To illustrate the definition of PAC-learnability, let us consider some concrete examples.

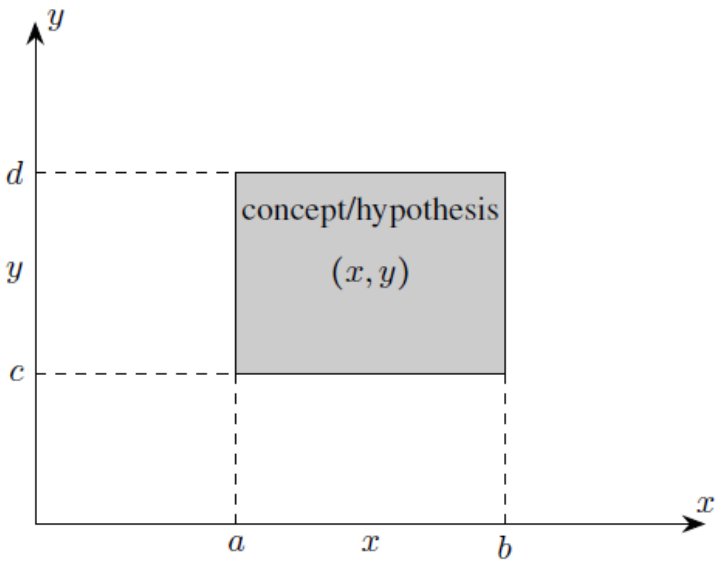


Figure : An axis-aligned rectangle in the Euclidean plane

Example

- Let the instance space be the set X of all points in the Euclidean plane. Each point is represented by its coordinates (x; y). So, the dimension or length of the instances is 2.
- Let the concept class C be the set of all "axis-aligned rectangles" in the plane; that is, the set of all rectangles whose sides are parallel to the coordinate axes in the plane (see Figure).
- Since an axis-aligned rectangle can be defined by a set of inequalities of the following form having four parameters

  $$a \le x \le b, \quad c \le y \le d$$

  the size of a concept is 4.
- We take the set H of all hypotheses to be equal to the set C of concepts, H = C.

Given a set of sample points labeled positive or negative, let L be the algorithm which outputs the hypothesis defined by the axis-aligned rectangle which gives the tightest fit to the positive examples (that is, that rectangle with the smallest area that includes all of the positive examples and none of the negative examples) (see Figure bleow).

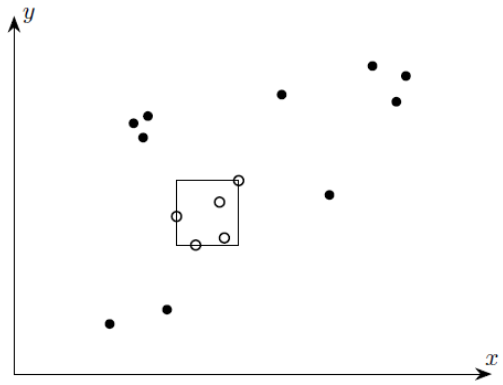


Figure : Axis-aligned rectangle which gives the tightest fit to the positive examples

It can be shown that, in the notations introduced above, the concept class C is PAC-learnable by the algorithm L using the hypothesis space H of all axis-aligned rectangles.

1.9 Vapnik-Chervonenkis (VC) dimension

The concepts of Vapnik-Chervonenkis dimension (VC dimension) and probably approximate correct (PAC) learning are two important concepts in the mathematical theory of learnability and hence are mathematically oriented. The former is a measure of the capacity (complexity, expressive power, richness, or flexibility) of a space of functions that can be learned by a classification algorithm. It was originally defined by Vladimir Vapnik and Alexey Chervonenkis in 1971. The latter is a framework for the mathematical analysis of learning algorithms. The goal is to check whether the probability for a selected hypothesis to be approximately correct is very high. The notion of PAC
learning was proposed by Leslie Valiant in 1984.

V-C dimension

Let H be the hypothesis space for some machine learning problem. The Vapnik-Chervonenkis dimension of H, also called the VC dimension of H, and denoted by V C(H), is a measure of the complexity (or, capacity, expressive power, richness, or flexibility) of the space H. To define the VC dimension we require the notion of the shattering of a set of instances.

Shattering of a set

Let D be a dataset containing N examples for a binary classification problem with class labels 0 and 1. Let H be a hypothesis space for the problem. Each hypothesis h in H partitions D into two disjoint subsets as follows:

$$\{x \in D \mid h(x) = 0\} \text{ and } \{x \in D \mid h(x) = 1\}.$$

Such a partition of S is called a "dichotomy" in D. It can be shown that there are 2^N possible dichotomies in D. To each dichotomy of D there is a unique assignment of the labels "1" and "0" to the elements of D. Conversely, if S is any subset of D then, S defines a unique hypothesis h as follows:

$$h(x) = \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{otherwise} \end{cases}$$

Thus to specify a hypothesis h, we need only specify the set {x Є D | h(x) = 1}. Figure 3.1 shows all possible dichotomies of D if D has three elements. In the figure, we have shown only one of the two sets in a dichotomy, namely the set {x Є D | h(x) = 1}.The circles and ellipses represent such sets.

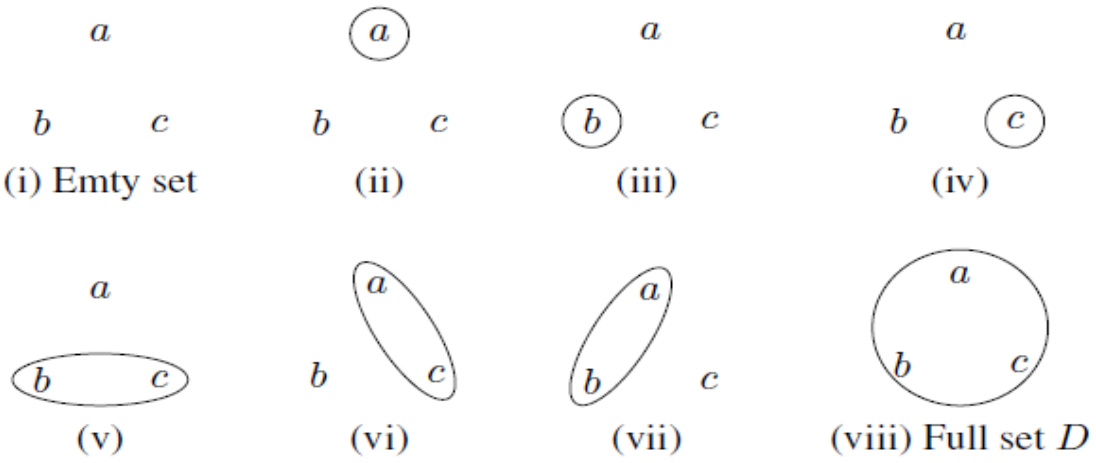


Figure 3.1: Different forms of the set $\{x \in S : h(x) = 1\}$ for $D = \{a, b, c\}$

Definition

A set of examples D is said to be shattered by a hypothesis space H if and only if for every dichotomy of D there exists some hypothesis in H consistent with the dichotomy of D.

The following example illustrates the concept of Vapnik-Chervonenkis dimension.

Example

In figure , we see that an axis-aligned rectangle can shatter four points in two dimensions. Then VC(H), when H is the hypothesis class of axis-aligned rectangles in two dimensions, is four. In calculating the VC dimension, it is enough that we find four points that can be shattered; it is not necessary that we be able to shatter any four points in two dimensions.

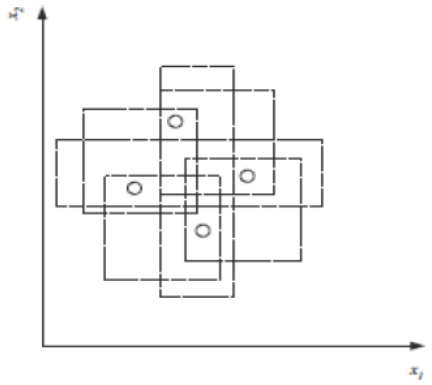


Fig: An axis-aligned rectangle can shattered four points. Only rectangle covering two points are shown.

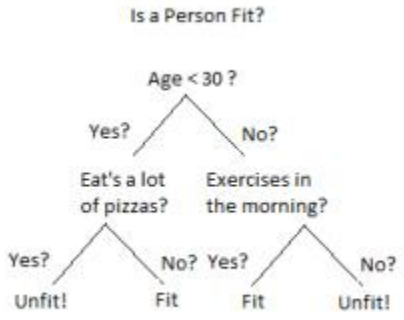
VC dimension may seem pessimistic. It tells us that using a rectangle as our hypothesis class, we can learn only datasets containing four points and not more.

Unit II
Supervised and Unsupervised Learning

Topics: Decision Trees: ID3, Classification and Regression Trees, Regression: Linear Regression, Multiple Linear Regression, Logistic Regression, Neural Networks: Introduction, Perception, Multilayer Perception, Support Vector Machines: Linear and Non-Linear, Kernel Functions, K Nearest Neighbors. Introduction to clustering, K-means clustering, K-Mode Clustering.

2.1. Decision Tree

Introduction Decision Trees are a type of Supervised Machine Learning (that is you explain what the input is and what the corresponding output is in the training data) where the data is continuously split according to a certain parameter. The tree can be explained by two entities, namely **decision nodes and leaves**. The leaves are the decisions or the final outcomes. And the decision nodes are where the data is split.



An example of a decision tree can be explained using above binary tree. Let's say you want to predict whether a person is fit given their information like age, eating habit, and physical activity, etc. The

decision nodes here are questions like 'What's the age?', 'Does he exercise?', and 'Does he eat a lot of pizzas'? And the leaves, which are outcomes like either 'fit', or 'unfit'. In this case this was a binary classification problem (a yes no type problem). There are two main types of Decision Trees:

1. **Classification trees** (Yes/No types)

What we have seen above is an example of classification tree, where the outcome was a variable like 'fit' or 'unfit'. Here the decision variable is **Categorical**.

2. **Regression trees** (Continuous data types)

Here the decision or the outcome variable is **Continuous**, e.g. a number like 123. **Working** Now that we know what a Decision Tree is, we'll see how it works internally. There are many algorithms out there which construct Decision Trees, but one of the best is called as **ID3 Algorithm**. ID3 Stands for **Iterative Dichotomiser 3**. Before discussing the ID3 algorithm, we'll go through few definitions. **Entropy** Entropy, also called as Shannon Entropy is denoted by H(S) for a finite set S, is the measure of the amount of uncertainty or randomness in data.

$$H(S) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)}$$

Intuitively, it tells us about the predictability of a certain event. Example, consider a coin toss whose probability of heads is 0.5 and probability of tails is 0.5. Here the entropy is the highest possible, since there's no way of determining what the outcome might be. Alternatively, consider a coin which has heads on both the sides, the entropy of such an event can be predicted perfectly since we know beforehand that it'll always be heads. In other words, this event has **no randomness** hence it's entropy is zero. In particular, lower values imply less uncertainty while higher values imply high uncertainty. **Information Gain** Information gain is also called as Kullback-Leibler divergence denoted by IG(S,A) for a set S is the effective change in entropy after deciding on a particular attribute A. It measures the relative change in entropy with respect to the independent variables

$$IG(S, A) = H(S) - H(S, A)$$

Alternatively,

$$IG(S, A) = H(S) - \sum_{i=0}^{n} P(x) * H(x)$$

where IG(S, A) is the information gain by applying feature A. H(S) is the Entropy of the entire set, while the second term calculates the Entropy after applying the feature A, where P(x) is the probability of event x. Let's understand this with the help of an example Consider a piece of data collected over the course of 14 days where the features are Outlook, Temperature, Humidity, Wind and the outcome variable is whether Golf was played on the day. Now, our job is to build a predictive model which takes in above 4 parameters and predicts whether Golf will be played on the day. We'll build a decision tree to do that using **ID3 algorithm.**

Day	Outlook	Temperature	Humidity	Wind	Play Golf
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes

D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

2.1.1 ID3

ID3 Algorithm will perform following tasks recursively

1. Create root node for the tree
2. If all examples are positive, return leaf node 'positive'
3. Else if all examples are negative, return leaf node 'negative'
4. Calculate the entropy of current state H(S)
5. For each attribute, calculate the entropy with respect to the attribute 'x' denoted by H(S, x)
6. Select the attribute which has maximum value of IG(S, x)
7. Remove the attribute that offers highest IG from the set of attributes
8. Repeat until we run out of all attributes, or the decision tree has all leaf nodes.

Now we'll go ahead and grow the decision tree. The initial step is to calculate H(S), the Entropy of the current state. In the above example, we can see in total there are 5 No's and 9 Yes's.

Yes	No	Total
9	5	14

$$Entropy(S) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)}$$

$$Entropy(S) = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right)$$

$$= 0.940$$

where 'x' are the possible values for an attribute. Here, attribute 'Wind' takes two possible values in the sample data, hence x = {Weak, Strong} we'll have to calculate:

1. $H(S_{weak})$
2. $H(S_{strong})$
3. $P(S_{weak})$
4. $P(S_{strong})$
5. $H(S) = 0.94$ which we had already calculated in the previous example

Amongst all the 14 examples we have **8 places where the wind is weak and 6 where the wind is Strong**.

Wind = Weak	Wind = Strong	Total
8	6	14

$$P(S_{weak}) = \frac{Number\ of\ Weak}{Total}$$
$$= \frac{8}{14}$$
$$P(S_{strong}) = \frac{Number\ of\ Strong}{Total}$$
$$= \frac{6}{14}$$

Now out of the 8 Weak examples, 6 of them were 'Yes' for Play Golf and 2 of them were 'No' for 'Play Golf'. So, we have,

$$Entropy(S_{weak}) = -\left(\frac{6}{8}\right)\log_2\left(\frac{6}{8}\right) - \left(\frac{2}{8}\right)\log_2\left(\frac{2}{8}\right)$$
$$= 0.811$$

Similarly, out of 6 Strong examples, we have **3 examples where the outcome was 'Yes' for Play Golf and 3 where we had 'No' for Play Golf**.

$$Entropy(S_{strong}) = -\left(\frac{3}{6}\right)\log_2\left(\frac{3}{6}\right) - \left(\frac{3}{6}\right)\log_2\left(\frac{3}{6}\right)$$
$$= 1.000$$

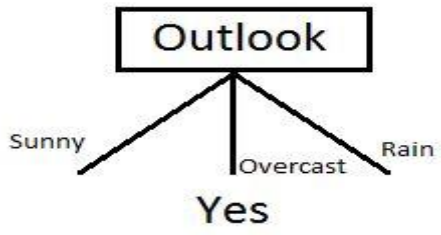
Remember, here half items belong to one class while other half belong to other. Hence we have perfect randomness. Now we have all the pieces required to calculate the Information Gain,

$$IG(S, Wind) = H(S) - \sum_{i=0}^{n} P(x) * H(x)$$
$$IG(S, Wind) = H(S) - P(S_{weak}) * H(S_{weak}) - P(S_{strong}) * H(S_{strong})$$
$$= 0.940 - \left(\frac{8}{14}\right)(0.811) - \left(\frac{6}{14}\right)(1.00)$$
$$= 0.048$$

Which tells us the Information Gain by considering 'Wind' as the feature and give us information gain of **0.048**. Now we must similarly calculate the Information Gain for all the features.

$$IG(S, Outlook) = 0.246$$
$$IG(S, Temperature) = 0.029$$
$$IG(S, Humidity) = 0.151$$
$$IG(S, Wind) = 0.048 \text{ (Previous example)}$$

We can clearly see that IG(S, Outlook) has the highest information gain of 0.246, **hence we chose Outlook attribute as the root node**. At this point, the decision tree looks like.



Here we observe that whenever the outlook is Overcast, Play Golf is always 'Yes', it's no coincidence by any chance, the simple tree resulted because of **the highest information gain is given by the attribute Outlook**. Now how do we proceed from this point? We can simply apply **recursion**, you might want to look at the algorithm steps described earlier. Now that we've used Outlook, we've got three of them remaining Humidity, Temperature, and Wind. And, we had three possible values of Outlook: Sunny, Overcast, Rain. Where the Overcast node already ended up having leaf node 'Yes', so we're left with two subtrees to compute: Sunny and Rain.

Next step would be computing $H(S_{sunny})$.

Table where the value of Outlook is Sunny looks like:

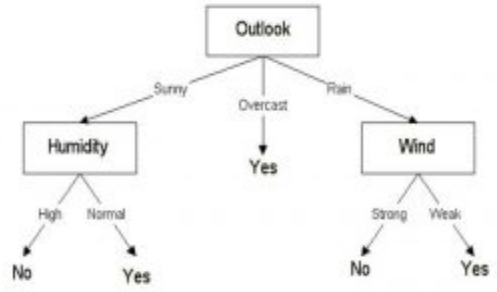
Temperature	Humidity	Wind	Play Golf
Hot	High	Weak	No
Hot	High	Strong	No
Mild	High	Weak	No
Cool	Normal	Weak	Yes
Mild	Normal	Strong	Yes

$$H(S_{sunny}) = \left(\frac{3}{5}\right)\log_2\left(\frac{3}{5}\right) - \left(\frac{2}{5}\right)\log_2\left(\frac{2}{5}\right) = 0.96$$

As we can see the **highest Information Gain is given by Humidity**. Proceeding in the same way with

$$S_{rain}$$

will give us Wind as the one with highest information gain. The final Decision Tree looks something like this. The final Decision Tree looks something like this.



2.1.2. Classification and Regression Trees

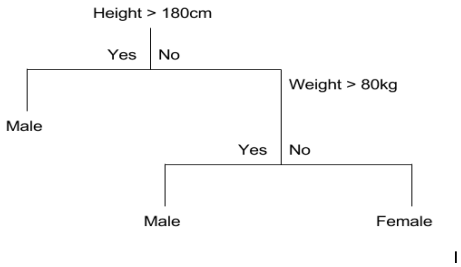
2.1.2.1. Classification Trees

A classification tree is an algorithm where the target variable is fixed or categorical. The algorithm is then used to identify the "class" within which a target variable would most likely fall.

An example of a classification-type problem would be determining who will or will not subscribe to a digital platform; or who will or will not graduate from high school.

These are examples of simple binary classifications where the categorical dependent variable can assume only one of two, mutually exclusive values. In other cases, you might have to predict among a number of different variables. For instance, you may have to predict which type of smartphone a consumer may decide to purchase.

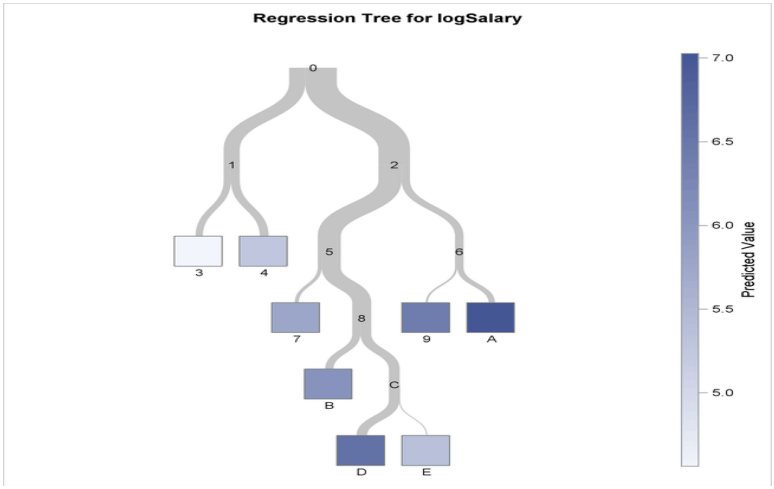
In such cases, there are multiple values for the categorical dependent variable. Here's what a classic classification tree looks like



2.1.2.2. Regression Trees

A regression tree refers to an algorithm where the target variable is and the algorithm is used to predict it's value. As an example of a regression type problem, you may want to predict the selling prices of a residential house, which is a continuous dependent variable.

This will depend on both continuous factors like square footage as well as categorical factors like the style of home, area in which the property is located and so on.



When to use Classification and Regression Trees

Classification trees are used when the dataset needs to be split into classes which belong to the response variable. In many cases, the classes Yes or No.

In other words, they are just two and mutually exclusive. In some cases, there may be more than two classes in which case a variant of the classification tree algorithm is used.

Regression trees, on the other hand, are used when the response variable is continuous. For instance, if the response variable is something like the price of a property or the temperature of the day, a regression tree is used.

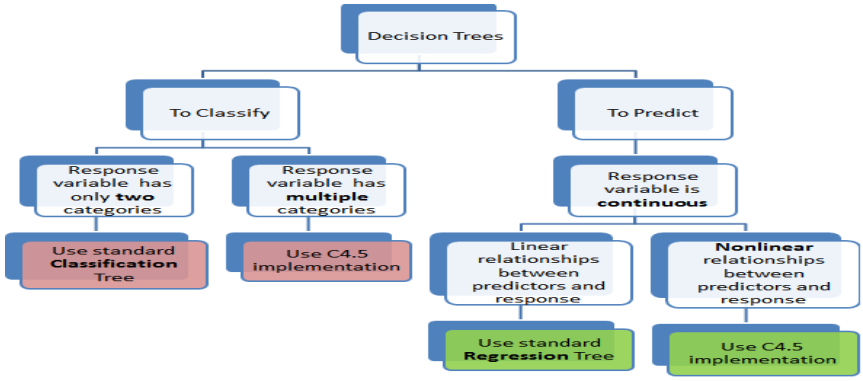
In other words, regression trees are used for prediction-type problems while classification trees are used for classification-type problems.

How Classification and Regression Trees Work

A classification tree splits the dataset based on the homogeneity of data. Say, for instance, there are two variables; income and age; which determine whether or not a consumer will buy a particular kind of phone.

If the training data shows that 95% of people who are older than 30 bought the phone, the data gets split there and age becomes a top node in the tree. This split makes the data "95% pure". Measures of impurity like entropy or Gini index are used to quantify the homogeneity of the data when it comes to classification trees.

In a regression tree, a regression model is fit to the target variable using each of the independent variables. After this, the data is split at several points for each independent variable.
At each such point, the error between the predicted values and actual values is squared to get "A Sum of Squared Errors" (SSE). The SSE is compared across the variables and the variable or point which has the lowest SSE is chosen as the split point. This process is continued recursively.



Advantages of Classification and Regression Trees
The purpose of the analysis conducted by any classification or regression tree is to create a set of if-else conditions that allow for the accurate prediction or classification of a case.
(i) The Results are Simplistic
The interpretation of results summarized in classification or regression trees is usually fairly simple. The simplicity of results helps in the following ways.

- It allows for the rapid classification of new observations. That's because it is much simpler to evaluate just one or two logical conditions than to compute scores using complex nonlinear equations for each group.
- It can often result in a simpler model which explains why the observations are either classified or predicted in a certain way. For instance, business problems are much easier to explain with if-then statements than with complex nonlinear equations.

(ii) Classification and Regression Trees are Nonparametric & Nonlinear
The results from classification and regression trees can be summarized in simplistic if-then conditions. This negates the need for the following implicit assumptions.

- The predictor variables and the dependent variable are linear.
- The predictor variables and the dependent variable follow some specific nonlinear link function.
- The predictor variables and the dependent variable are monotonic.

Since there is no need for such implicit assumptions, classification and regression tree methods are well suited to data mining. This is because there is very little knowledge or assumptions that can be made beforehand about how the different variables are related.
As a result, classification and regression trees can actually reveal relationships between these variables that would not have been possible using other techniques.
(iii) Classification and Regression Trees Implicitly Perform Feature Selection
Feature selection or variable screening is an important part of analytics. When we use decision trees, the top few nodes on which the tree is split are the most important variables within the set. As a result, feature selection gets performed automatically and we don't need to do it again.
Limitations of Classification and Regression Trees